

Selenium with Python Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 2 Months

Level: Intermediate

Course page: <https://archerinfotech.in/courses/testing-qa/selenium-python-training-in-pune>

Course Overview

Selenium with Python is the fastest route from manual testing into automation for anyone who found Java a barrier. This 2-month programme starts with testing fundamentals, teaches Python from setup through object-oriented programming, then builds up Selenium 4 WebDriver, PyTest, and a Page Object Model framework wired to Jenkins — closing with a full module on AI-assisted testing. Designed for manual QA testers moving to automation and for graduates targeting Pune's QA pipeline.

Why Learn Selenium with Python in 2026

Selenium remains the most widely deployed browser-automation tool in the world, and Python has become its fastest-growing binding. The reason is practical rather than ideological: a manual tester can be productive in Python in weeks rather than months, and the same language then carries over to API testing with PyTest, data validation, and the scripting that QA work constantly demands. Pune's services sector — Persistent, Capgemini, LTIMindtree, Tech Mahindra, Cognizant, Wipro — hires Selenium engineers at volume, and an increasing share of those requisitions now say 'Java or Python'.

What changed recently is the expectation attached to the role. A 2020 Selenium job description asked for WebDriver and TestNG. A 2026 one asks for a framework you designed, tests running in CI on every merge, reporting a manager can read, and — increasingly — evidence that you can use AI tooling to produce and maintain tests without shipping nonsense into the suite. This course is built to that specification: the framework module and the CI module are not appendices, and AI-assisted testing gets a full module rather than a closing mention.

Python's specific advantage in QA is leverage. PyTest fixtures make setup and teardown genuinely composable, parameterisation is a single decorator, and the plugin ecosystem covers parallelism, retries, HTML and Allure reporting without custom code. Teams that adopt Python for Selenium usually find their API tests migrate to PyTest too, which is exactly the breadth that turns a QA Automation Engineer into an SDET.

- Python is taught from zero — no programming background assumed
- Selenium 4 with the W3C protocol and modern waits, not Selenium 3 habits
- PyTest to production depth — fixtures, markers, parallel runs, plugins
- A framework you design, not a folder of scripts
- Jenkins CI and Allure reporting included, not left as self-study
- A full AI-assisted testing module, with validation discipline

Who This Course Is For

This course fits you if

- Manual QA testers with 1+ years of experience moving into automation

- BE, BCA, BSc-CS or MCA graduates targeting QA and SDET roles in Pune
- Testers who started Java, found it heavy going, and want a faster route in
- Python developers who want to move into test engineering
- Support or operations staff with scripting exposure looking to move into QA

This course is not the right fit if

- Anyone wanting a pure manual-testing course — take Software Testing & QA instead
- Engineers who need Java specifically because their target team is Java-only
- Anyone expecting to avoid writing code — this is a programming-led track
- Testers looking only for a certificate rather than a working framework and portfolio

Detailed Curriculum

Module 1: Software Testing Fundamentals (Week 1)

The grounding every automation engineer is assumed to have and is quietly judged on in interviews. What testing is for, SDLC and STLC as processes rather than acronyms, verification versus validation, and the functional and non-functional testing types.

Then the daily working vocabulary: smoke, sanity and regression testing and when each runs; writing test scenarios and test cases that someone else can execute; the defect life cycle; and severity versus priority, which candidates routinely conflate. Agile, Scrum and Jira close the module, since that is the process every Pune QA role actually runs inside.

- Introduction to software testing
- SDLC and STLC
- Testing levels and types
- Functional and non-functional testing
- Smoke, sanity and regression testing
- Test cases and test scenarios
- Defect management and the defect life cycle
- Severity and priority
- Agile testing and Scrum
- Jira for test management

Module 2: Python for Test Automation (Weeks 1–2)

Python from installation onward, scoped to what test automation actually uses. Setup and interpreter basics, variables and data types, operators, conditionals and loops, then the collection types — strings, lists, tuples, sets and dictionaries — which is where most test-data handling lives.

Functions, modules and packages follow, then classes, objects and the object-oriented programming that the Page Object Model depends on entirely. Exception handling and file handling cover reading test data and failing informatively. Virtual environments close the module so every student's project is reproducible.

- Python setup and the interpreter
- Variables, data types and operators
- Conditional statements and loops
- Strings and string methods
- Lists, tuples, sets and dictionaries
- Functions, arguments and return values
- Modules and packages
- Classes, objects and object-oriented programming
- Exception handling
- File handling
- Virtual environments and dependency isolation

Module 3: Selenium WebDriver with Python (Weeks 3–4)

The core automation skill. Selenium's architecture and the W3C WebDriver protocol, driver setup and browser automation, then locators in depth — id, name, class, link text, CSS selectors and XPath — with the discipline of choosing locators that survive a UI change rather than ones that merely work today.

The module then works through everything a real page throws at you: forms, dropdowns, alerts, frames, multiple windows and tabs, web tables, and mouse and keyboard actions. Synchronisation gets particular attention — implicit versus explicit waits, and why `time.sleep` is the first thing a reviewer will flag. JavaScript execution, screenshots and file upload and download close it.

- Selenium architecture and the W3C protocol
- WebDriver setup and browser automation
- Locators — id, name, class, link text
- XPath — absolute, relative, axes and functions
- CSS selectors
- Web elements and element interactions
- Forms, dropdowns, checkboxes and radio buttons
- Alerts, frames, windows and tabs
- Web tables and dynamic content
- Mouse and keyboard operations
- Explicit and implicit waits, and why sleep is not a wait
- JavaScript execution
- Screenshots, file upload and download

Module 4: PyTest (Weeks 4–5)

The test runner that makes Python's automation story compelling. Test discovery and naming conventions, plain assert statements with PyTest's introspection, and the fixture model — the feature that genuinely distinguishes PyTest from TestNG once you understand scope and composition.

Fixture scope (function, class, module, session) is taught against real cost: a browser launched per test versus per session changes suite runtime by an order of magnitude. Parameterisation, markers for selective runs, test classes, `conftest.py` for shared fixtures, parallel execution with `pytest-xdist`, and the plugin ecosystem close the module.

- Introduction to PyTest and test discovery
- Test functions and naming conventions
- Assertions and assertion introspection
- Fixtures — setup and teardown done properly
- Fixture scope — function, class, module, session
- Parameterisation for data-driven tests
- Markers and selective execution
- Test classes and grouping
- `conftest.py` and shared fixtures
- Parallel testing with `pytest-xdist`
- PyTest plugins and PyTest reports

Module 5: Automation Framework Development (Weeks 5–6)

The module that separates someone who writes tests from someone who owns a suite. The Page Object Model implemented properly — page classes, locators as class attributes, methods that express user intent rather than clicks — plus base classes carrying shared driver behaviour.

Then the surrounding structure: reusable utilities, configuration files and environment management, test-data management and data-driven testing, logging that makes a CI failure diagnosable, error handling, and screenshot capture on failure. Folder structure is treated as a design decision, because a framework a new joiner cannot navigate is a framework that gets rewritten.

- Page Object Model design
- Base classes and shared driver behaviour

- Reusable utilities and helper modules
- Configuration files and environment management
- Test data management
- Data-driven testing
- Logging for diagnosable CI failures
- Error handling and custom exceptions
- Screenshot handling on failure
- Framework folder structure as a design decision

Module 6: BDD with Behave (Week 6)

Behaviour-driven development as it is actually practised, including an honest account of when it earns its overhead and when it is ceremony. BDD fundamentals and the three-amigos idea, then Gherkin syntax — Given, When, Then — and feature files written to be readable by someone who is not a programmer.

Scenarios and scenario outlines for data variation, step definitions that map plain language to Python, hooks for setup and teardown, and tags for selective execution. The module ends by integrating Behave with the Selenium framework built in the previous module, so BDD is a layer on top rather than a parallel universe.

- BDD fundamentals and the three-amigos practice
- Gherkin syntax — Given, When, Then
- Feature files written for non-programmers
- Scenarios and scenario outlines
- Step definitions and step reuse
- Hooks for setup and teardown
- Tags and selective execution
- Integrating Behave with the Selenium framework
- When BDD earns its overhead, and when it does not

Module 7: Reporting (Week 7)

Reports are how a QA engineer communicates with people who will never read the code, so they are treated as a deliverable rather than a switch. PyTest HTML reports for the quick local view, then Allure — the report format most Pune teams standardise on — with suites, steps, attachments and history.

Screenshot and log attachment on failure turns a red test into an actionable one. The module also covers custom reporting, environment and build metadata in the report, and the practical question of what a test-execution summary should say to a delivery manager versus a developer.

- PyTest HTML reports
- Allure reports — suites, steps and history
- Attaching screenshots on failure
- Attaching logs and request data
- Environment and build metadata
- Custom reporting and report customisation
- Writing a summary a delivery manager can read

Module 8: Git and GitHub (Week 7)

Version control as the automation engineer uses it. Git fundamentals — the working tree, staging and commits — then the commands that matter day to day: status, diff, log, add, commit, push and pull.

Branching and merging with conflict resolution done without losing work, then the GitHub layer: repositories, pull requests, code review on test code, and project management. The module closes on maintaining an automation project as a shared asset — .gitignore for drivers and reports, meaningful commit messages, and a README that lets a reviewer run your suite.

- Version control fundamentals
- Core Git commands — status, diff, log, add, commit

- Repository management and remotes
- Branching and merging
- Conflict resolution without losing work
- Pull requests and code review on test code
- GitHub project management
- gitignore, commit hygiene and a runnable README

Module 9: CI/CD with Jenkins (Week 8)

Getting the suite running without you. CI/CD fundamentals and why an automation suite that only runs on the author's laptop delivers a fraction of its value. Jenkins setup, job creation, and running PyTest through Jenkins with the right Python environment.

GitHub integration triggers runs on push and on pull request; scheduled builds cover nightly regression. Test-report publishing surfaces Allure output in Jenkins itself, and the module ends on pipeline fundamentals plus the operational reality of a suite that fails at 2am — notifications, retries and triage.

- CI/CD fundamentals for test automation
- Jenkins setup and job creation
- Running PyTest through Jenkins
- Managing the Python environment on an agent
- GitHub integration and build triggers
- Scheduled builds for nightly regression
- Publishing test reports in Jenkins
- Pipeline fundamentals
- Notifications, retries and failure triage

Module 10: AI-Assisted Testing (Week 8 + capstone)

The capability Pune QA panels now probe for directly, taught with the validation discipline that makes it professional rather than reckless. Generative AI applied across the testing workflow: producing test scenarios from requirements, generating test cases, and turning manual test cases into runnable PyTest automation.

Then the code-level uses — generating Selenium Python scripts, locator suggestions, page objects and framework scaffolding — followed by the analytical ones: AI-assisted debugging, code explanation and refactoring, documentation generation, defect classification, and log and error analysis. The module is explicit throughout that an AI-generated test is a draft: it closes on prompt engineering for QA, coding assistants in an automation workflow, and how to validate output before it enters a suite that other people trust.

- Generative AI for software testing — where it helps and where it does not
- AI-based test scenario generation
- Test case generation from requirements
- Generating Selenium Python scripts using AI
- AI-assisted locator generation
- Converting manual tests into PyTest automation
- Synthetic test data generation
- AI-assisted debugging
- Code explanation and refactoring
- Automated documentation generation
- Defect classification using AI
- Log and error analysis
- AI-assisted framework generation
- Prompt engineering for QA professionals
- Coding assistants for Python automation
- Responsible use and validation of AI-generated tests

Capstone Projects

E-commerce Regression Suite with Page Object Model

A complete PyTest and Selenium framework automating the critical path of a public e-commerce demo site — search, product selection, cart, address, payment-step validation and order confirmation. Page Object Model architecture, data-driven across multiple product categories, Allure reporting with screenshots on failure, and a Jenkins job that runs it nightly. The portfolio piece that reads best on a QA fresher CV.

- Technology: Selenium 4
- Technology: Python
- Technology: PyTest
- Technology: Page Object Model
- Technology: Allure
- Technology: Jenkins
- Technology: Git

Multi-Role Banking Application Framework

A harder framework exercising multi-window handling, alerts, frames and web tables, with parameterised runs across three user roles and role-specific permission verification. Configuration-driven environment switching, structured logging, and a failure notification hook. Demonstrates that you can handle the messy pages real applications actually have.

- Technology: Selenium 4
- Technology: Python
- Technology: PyTest
- Technology: Behave
- Technology: Logging
- Technology: Config management
- Technology: Jenkins

Capstone — BDD Suite with CI and AI-Assisted Maintenance

The capstone combines a Behave BDD layer over the Page Object framework, cross-browser execution, parallel runs with pytest-xdist, Allure reporting published in Jenkins, and a documented AI-assisted workflow — scenarios drafted with AI, then reviewed, corrected and validated by you, with the review notes kept in the repository. Pushed to GitHub with a README a reviewer can follow.

- Technology: Selenium 4
- Technology: Python
- Technology: PyTest
- Technology: pytest-xdist
- Technology: Behave
- Technology: Allure
- Technology: Jenkins
- Technology: GitHub
- Technology: AI coding assistants

Career Outcomes

Pune's QA automation market splits into two bands. The services pipeline — Persistent, Capgemini, LTIMindtree, Tech Mahindra, Cognizant, Wipro, Accenture, IBM India — hires automation engineers at scale, typically through a bench-to-project ramp; fresher offers start around Rs. 3.5–5 LPA and a working framework portfolio moves the number. Product-company SDET roles at Cybage, BrowserStack, Druva, Helpshift and similar pay meaningfully more for the same headline experience, because the expectation includes CI ownership, API testing and occasional development work — which is exactly what the framework, CI and AI modules prepare you for.

Python-specific demand has grown fastest where a team wants one language across UI automation, API testing and test data work. That breadth is the practical route from QA Automation Engineer to SDET, and it is why this course teaches PyTest to a depth that carries over to API automation rather than stopping at browser scripts. We do not claim guaranteed

placement: our institute-records placement rate is 90% across all tracks, and this track runs in that range.

Roles after this course

- QA Automation Engineer
- Selenium Python Automation Engineer
- SDET (Software Development Engineer in Test)
- Test Automation Lead
- API + UI Automation Engineer

Indicative salary bands

- QA Automation Engineer (fresher) - Rs. 3.5–5 LPA (services) / Rs. 4.5–6 LPA (product) (source: AmbitionBox Pune QA Automation (last 12 mo))
- QA Automation Engineer (1–3 yrs) - Rs. 6–10 LPA (source: Indeed Pune QA Automation listings (last 12 mo))
- SDET (3–6 yrs) - Rs. 10–18 LPA (source: Glassdoor Pune SDET)
- Test Architect / Sr SDET (6+ yrs) - Rs. 14–22 LPA (source: AmbitionBox Pune Test Architect)

Companies hiring for these skills

- Persistent Systems
- Capgemini
- LTIMindtree
- Tech Mahindra
- Cognizant
- Wipro
- Accenture
- Cybage
- BrowserStack
- Druva
- Helpshift
- Saksoft
- TCS
- IBM India
- Amdocs

Batch Modes and Duration

Duration: 2 months (8 weeks) for the weekday and online tracks; about 10 weeks for the weekend track

Classroom

- Location: Archer Infotech Kothrud campus (Flat No. 12, Divyadarshan Housing Society, Kothrud, Pune 411038)
- Morning batch: Monday–Friday 10:00–11:30
- Evening batch: Monday–Friday 19:00–20:30
- Saturday lab session: 10:00–13:00 (three-hour deep practice)

Live online

- Live sessions: Monday–Friday 19:30–21:00 IST
- Recordings available in the LMS within 24 hours
- Google Meet for live sessions
- GitHub for code distribution
- Batch Slack channel for doubts

Weekend

- Saturday and Sunday 10:00–13:00 (3 hours per day, 6 hours per week)
- The weekend track runs about 10 weeks instead of 8 to keep total contact hours the same

Batch sizes stay under 18 for weekday and online tracks and under 12 at weekends — small enough that the trainer knows your project repository. New batches typically start every three to four weeks; check the live batch schedule.

Fees

The Selenium with Python track is priced in the mid band of our catalogue, reflecting the two-month duration and the per-batch lab and Jenkins infrastructure. EMI plans are available; contact admissions for the current fee structure.

Indicative range: Rs. 25,000 – Rs. 35,000 (typical track band)

- One-time payment (5% discount)
- EMI: 50% at enrolment, 50% at four weeks
- EMI: three-month plan

Placement Support

Placement support is bundled with the course, with no separate fee. Graduates plug into the same Archer placement cell that supports our Java, Python and Full Stack tracks, built on seventeen years of relationships with Pune hiring managers. The process starts in Week 6, in parallel with your capstone, so your CV is reviewed and your GitHub portfolio is presentable by the time you finish rather than weeks afterwards.

We do not guarantee placement. Our institute-records placement rate is 90% and this track runs in that range; the variance is explained far more often by interview communication than by framework knowledge. The placement cell expects you to apply actively in the sixty-day window after completion — we open doors and prepare you, but you walk through them.

The placement process

- Week 6: CV review, GitHub portfolio audit and resume formatting for services and product tracks
- Week 7: First mock interview (technical) with structured feedback
- Week 8: Second mock interview (HR and communication) plus an offer-negotiation primer
- Post-completion weeks 1–2: Direct introductions to partner companies hiring automation engineers
- Post-completion weeks 3–8: Weekly placement-cell check-ins and interview coaching as you apply

Partner companies

- Persistent Systems
- Capgemini
- LTIMindtree
- Tech Mahindra
- Amdocs
- Cybage
- Cognizant
- Wipro
- Saksoft
- TCS

Prerequisites and How to Start

The course assumes you understand what testing is for and can navigate a computer confidently. It does not assume any Python syntax, any Selenium knowledge, or any framework experience. A typical batch is roughly 40% manual QA testers with a year or more of experience, 35% recent graduates, and 25% people moving across from support, operations or development. The pace adjusts naturally — manual testers usually need more support through the Python module and then accelerate through framework design, while graduates pick up Python quickly and need more help with framework patterns.

None of the starting steps below are gated. Doing them before the first session means you spend that session learning rather than installing.

Suggested first steps

- Install Python 3.12 or later and confirm it with `python --version`

- Install Visual Studio Code or PyCharm Community Edition — either is fine for this course
- Create a free GitHub account and practise the clone, commit and push cycle once
- Read the Selenium introduction on selenium.dev for about fifteen minutes so the first session has context
- Work through any free Python basics tutorial as far as loops and lists — enough to recognise the syntax

Frequently Asked Questions

Do I need any programming experience before joining?

No. Module 2 teaches Python from installation through object-oriented programming, which is everything the rest of the course needs. Manual testers with no coding background make up a large share of every batch.

Should I learn Selenium with Python or with Java?

Java has slightly more Pune listings because of existing services-sector frameworks. Python gets you productive faster and carries into API testing and data work. If you are switching from manual testing and want momentum, choose Python; if you are targeting a specifically Java team, choose Java.

Is PyTest better than TestNG?

They solve the same problem in different languages. PyTest's fixtures compose more flexibly and its parameterisation is terser; TestNG has richer built-in suite XML. This course teaches PyTest to production depth including parallel execution and plugins.

How much of the course is actually AI-related?

One full module of sixteen topics, plus AI-assisted work threaded through the capstone. It covers generating scenarios, scripts, locators and page objects, debugging and log analysis, and — importantly — how to validate AI output before it enters a suite others depend on.

Will I finish with something I can show an interviewer?

Yes. You build three projects, ending with a capstone framework on GitHub that runs in Jenkins with Allure reporting. Interviewers ask to see a repository far more often than they ask for a certificate.

Can I take this if I already did the Selenium with Java course?

You can, and some students do, but it is usually not the best use of your time. The framework concepts transfer directly. Adding API Testing and Automation or Playwright with TypeScript broadens your profile more than repeating Selenium in a second language.

Do you cover mobile automation with Appium?

No. This track stays focused on web automation with Selenium. Appium is a separate specialisation, and we would rather teach one framework to production depth than two superficially.

What is the batch size and how often do batches start?

Under 18 for weekday and online batches, under 12 at weekends. New batches typically start every three to four weeks across weekday, weekend and live-online formats.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.